1

If our bipartite graph is split into our vertex sets V_1, V_2 with only edges between vertices in V_1 to vertices in V_2. We can find a maximum bipartite matching for this graph by copying our bipartite graph with the same edges but making them directed from V_1 to V_2. Next we add a node s which connects to all nodes in V_1 with each edge having a capacity of 1. Subsequently we add an edge from every vertex in V_2 to a node t with each edge here also having a capacity of 1. Lastly we induce a flow by adding an edge with infinite capacity from t to s which has a cost of -1 and every other edge has a 0 cost.

All nodes start with 0 supply and the optimal flow would be equal in magnitude to the size of the maximal matching (magnitude because our solution flow would have a negative cost).

Thus our optimal flow can be used to find the maximal matching by looking at which edges in G' have flows and the edges that were in the original G that have flows would be vertex disjoint and in our maximal match.

Proof of correctness

Let $F(G')$ denote the cost optimal flow on our graph G' described above and let $|M(G)|$ be the size of the maximum matching. Prove that $F(G') = |M(G)|$

Given our optimal matching $M(G)$, we know this is a vertex disjoint set of edges in G and is the maximum size of any set of such vertex disjoint edges. Our optimal solution of $F(G')$ can use this set of edges in its optimal flows. Since the only negative edge (edge with any cost really) is from t to s we know that we must use this edge as much as we can.

Suppose we take this edge with a flow of $|M(G)|$ (which is now our current $F(G')$), then s currently has excess and t has deficit of $|M(G)|$. Thus we split the supply in s to the vertices that are part of V_1 and in our maximal matching. If s still has supply left or is now in deficit then this creates a contradiction because it would imply that $|M(G)|$ is not equal to the number of V_1 vertices in the maximal matching, which is not possible because the number of vertices in a matching is 2 times the number of edges in the matching, and since each edge has one vertex in V_1 and one in V_2 it should be split 50/50 so $|M(G)|$ in V_1 same amount in V_2. We know it is 50/50 because all edges are from V_1 to V_2.

We now have a set of vertices of size $|M(G)|$ in V_1 that have a 1 supply, and we now flow from these nodes to nodes in V_2 along the edges in the maximal matching. So now all nodes in V_1 would have a supply of 0, there is flow along the edges from V_1 to V_2 that correspond to the maximal matching and now nodes in V_2 have a combined supply of $|M(G)|$. This has to be the case following the similar logic of all edges being from V_1 to V_2 in the previous paragraph. Thus we take the edges from the nodes in V_2 to t making every node have a supply of 0 but now there is a flow where the total cost is $-|M(G)|$. But how do we know that two nodes in V_1 both had a flow to the same node in V_2 causing an arc from V_2 to t to saturate at 1 but

leaving a node in V_2 with some supply. That would create a contradiction because it would imply that our matching is not vertex disjoint so that cannot happen.

How do we know there is not a more optimal flow that minimizes the cost further? Well that would imply there is some way to add a new flow from an unused vertex in V_1 to an unused vertex in V_2, which creates a contradiction because then this edge could be added to our match and still be vertex disjoint meaning we have a matching that is larger than our maximal matching.

Therefore $F(G') = |M(G)|$

2

Suppose we are trying to find the number of vertex disjoint paths from u to v in G, we can construct our G' as follows. For any vertex x in G that is not u or v, we essentially "split" it into x_{in} and x_{out}. So all of f the incoming edges into x point to x_{in} and all outgoing edges from x leave from x_{out} and we create an arc between x_{in} and x_{out} with a capacity of 1.

> Quick sidenote this works for undirected graphs as well because there are no symmetric arcs with this configuration!

u and v can keep the rest of their edges and do not need splittig and instead this can be our s and t where u is s and v is t. Then we can either do a negative arc from t to s with infinite capacity and a cost of -1 to find our optimal solution giving us the magnitude of our optimal cost being the number of vertex disjoint paths OR we could have s have a supply of k and t have a deficit of k and iterate k until there is no more feasible solution.

> We can also do this same k strategy for 1 but I like the negative edge better.

Proof of correctness.

Well we know we use every vertex at most once because of the arc from x_{in} to x_{out} with a capacity of 1. Meaning if two different paths used the same vertex it would mean that x_{in} would end with a supply of 1 and x_{out} would have a supply of -1 which is not possible for a feasible flow. Thus we know that each path is vertex distinct in G' but how do we know a MCNF finds an optimal solution.

Suppose the MCNF terminates at a k smaller than the maximum number of vertex disjoint paths such that $k+1$ finds no feasible solution. Well our MCNF could have just taken the $|P|$ paths vertex disjoint paths from Mengers theorem and would have been able to take $|P|$ units of flow from s to t with only 1 unit of flow per path (alligning with the 1 flow capacity limits on the intermediate arcs). Since $k < |P|$ we could find a $k+1$ supply solution by using a subset of the paths showing that we are actually able to find a k that is at least as large as $|P|$

Suppose the MCNF either terminates at a k larger than the maximum number of vertex disjoint paths from Mengers theorem. This creates a contradiction because then our MCNF has found k distinct/vertex disjoint ways/paths from s (u) to t (v). Thus we can find a new max paths set by taking all the paths with flows on them (with the contractions between any x_{in} with the corresponding x_{out}). This creates a contradiction because we assumed we had a maximum vertex disjoint paths set.

Therefore $k = |P|$

3

Prove $cap(S \cup T, \overline{S \cup T}) + cap(S \cap T, \overline{S \cap T}) \leq cap(S, \overline{S}) + cap(T, \overline{T})$ given that there are no edges from a node in S to another node in S (and same with T).

Since there are no edges within S and T, the right side of our equation is the total number of edges that leave S plus the total number of edges that leave T (cap does not care about incoming edges). The left side can only contain outgoing arcs from $S \cap T$ or $S \cup T$ implying that we only having outgoing arcs from S and from T.

Suppose that $\exists S, T \subseteq V(G), s.t., cap(S \cup T, \overline{S \cup T}) + cap(S \cap T, \overline{S \cap T}) > cap(S, \overline{S}) + cap(T, \overline{T})$. This implies that the left side has more outgoing arcs then the outgoing arcs from S and T combined. Well we know that if S and T are disjoint then $cap(S \cap T, \overline{S \cap T})$ is 0 and $cap(S \cup T, \overline{S \cup T})$ is equal to the number of outgoing edges from S and T minus the ones that are from S with a head in T and vice versa. Thus S and T cannot be disjoint because then we have $cap(S \cup T, \overline{S \cup T}) + cap(S \cap T, \overline{S \cap T}) \leq cap(S, \overline{S}) + cap(T, \overline{T})$. Suppose then $S = T$, then $S = S \cap T = S \cup T = T$ then we have $cap(S \cup T, \overline{S \cup T}) + cap(S \cap T, \overline{S \cap T}) = cap(S, \overline{S}) + cap(T, \overline{T})$ which is not greater than so this does not work either.

Thus $\exists U = S \cap T$ and $S \neq T$. Since there are no edges between nodes within S or T, then $cap(S, \overline{S}) = cap(U, \overline{U}) + cap(S - U, \overline{S - U})$ with the same logic for T. Thus we get the following simplifications:

$$cap(S \cup T, \overline{S \cup T}) + cap(S \cap T, \overline{S \cap T}) > cap(S, \overline{S}) + cap(T, \overline{T})$$

$$cap(S \cup T, \overline{S \cup T}) + cap(U, \overline{U}) > cap(S - U, \overline{S - U}) + cap(T - U, \overline{T - U}) + 2cap(U, \overline{U})$$

$$cap(S \cup T, \overline{S \cup T}) > cap(S - U, \overline{S - U}) + cap(T - U, \overline{T - U}) + cap(U, \overline{U})$$

This last statement cannot be true because even if S and T have no outgoing edges to eachother, it would imply that there are more outgoing edges in $S \cup T$ then its subparts of $T - U, U, S - T$.

Therefore $cap(S \cup T, \overline{S \cup T}) + cap(S \cap T, \overline{S \cap T}) \leq cap(S, \overline{S}) + cap(T, \overline{T})$